

Ashtech Pay

Direct API — SDK Documentation

v1 • Mobile Money + Pay-In Crypto — catalogue actif

Initiez des paiements Mobile Money directement depuis votre serveur, sans redirection. Gerez les flux USSD Push, OTP USSD, OTP API fournisseur et Wave dans le catalogue actif.

ENDPOINT COLLECT
POST /v1/collect

AUTHENTICATION
Bearer YOUR_API_KEY

BASE URL
https://ashtechpay.top

VERSION
v1 — Août 2026

Sommaire

1. Introduction
2. Authentification
3. GET /v1/countries — Pays et operateurs
4. Pay-In Crypto — /v1/crypto/assets + /v1/crypto/collect
5. POST /v1/collect — Initier un paiement Mobile Money
6. Flux de paiement (USSD Push · OTP USSD · OTP API · Wave)
7. GET /v1/transaction/:id — Statut d'une transaction
8. GET /v1/fees — Grille tarifaire en temps reel
9. Webhooks
10. Codes d'erreur

1. Introduction

L'Ashtech Pay API unifie plusieurs passerelles de paiement africaines en une seule interface REST. Initiez des paiements Mobile Money dans le catalogue actif sans redirection. Le routage entre les operateurs est automatique — vous n'avez pas a choisir le fournisseur.

Caracteristique	Detail
Base URL	https://ashtechpay.top
Format	JSON uniquement — Content-Type: application/json
Authentification	Bearer token dans l'en-tete Authorization
Protocole	HTTPS obligatoire
Versioning	Prefixe /v1/ dans tous les endpoints

2. Authentification

Toutes les requetes doivent inclure votre cle API dans l'en-tete HTTP Authorization.

HTTP

```
Authorization: Bearer YOUR_API_KEY
```

Utilisez votre cle API uniquement depuis votre serveur (Node.js, Python, PHP...). Ne l'incluez jamais dans du code cote navigateur ou application mobile.

Exemple d'appel authentifie (Node.js) :

Node.js

```
const response = await fetch("https://ashtechpay.top/v1/collect", {
  method: "POST",
  headers: {
    "Authorization": "Bearer YOUR_API_KEY",
    "Content-Type": "application/json"
  },
  body: JSON.stringify({ /* ... */ })
});
```

3. Pays et operateurs — GET /v1/countries

Retourne la liste complete des pays actifs et leurs operateurs Mobile Money disponibles. Cette liste est geree par l'administrateur — tout ajout ou retrait de pays/operateur est immediatement visible via cet endpoint.

Node.js

```
fetch("https://ashtechpay.top/v1/countries", {
  headers: { "Authorization": "Bearer YOUR_API_KEY" }
})
```

curl

```
curl https://ashtechpay.top/v1/countries \
-H "Authorization: Bearer YOUR_API_KEY"
```

Reponse

json

```
[
  {
    "code": "CM",
    "name": "Cameroun",
    "currency": "XAF",
    "operators": ["MTN Money", "Orange Money"]
  },
  {
    "code": "SN",
    "name": "Senegal",
    "currency": "XOF",
    "operators": ["E-money", "Free Money", "Orange Money", "Wave Money"]
  }
  // ...
]
```

Pays disponibles (10)

Pays	Code	Devise	Operateurs
Benin	BJ	XOFB	Celtis Money, Coris Money, Moov Money, MTN Money
Burkina Faso	BF	XOFF	Moov Money, Orange Money (OTP), Wallet LigdiCash
Cameroun	CM	XAF	MTN Money, Orange Money
Cote d'Ivoire	CI	XOFC	Moov Money, MTN Money, Orange (OTP), Wave Money
Gabon	GA	XAFG	Airtel Money, Moov Money
Mali	ML	XOFM	Moov Money, Orange Money
Niger	NE	XOFN	Airtel Money
RD Congo	CD	CDF	Afri Money, Airtel, Mpesa Money, Orange, Vodacom
Senegal	SN	XOFS	E-money, Free Money, Orange Money (OTP), Wave Money
Togo	TG	XOFT	Flooz (Moov), T-Money

Legende : (OTP USSD) = code a composer pour recevoir l'OTP • (OTP API fournisseur) = SMS automatique gere par le fournisseur • Wave = lien de paiement Wave

4. Pay-In Crypto — /v1/crypto/assets + /v1/crypto/collect

Le Pay-In Crypto utilise la meme cle API ak_... que Mobile Money, mais des endpoints dedies afin de ne modifier aucun contrat existant. Les cryptos utilisent uniquement GET /v1/crypto/assets et POST /v1/crypto/collect ; /v1/collect reste reserve a Mobile Money. Commencez par recuperer les reseaux autorises, puis creez une adresse de depot unique.

GET /v1/crypto/assets retourne uniquement les reseaux crypto actifs et autorises. Utilisez la valeur asset_code retournee dans l'appel de creation.

GET /v1/crypto/assets

curl

```
curl https://ashtechpay.top/v1/crypto/assets \
-H "Authorization: Bearer YOUR_API_KEY"
```

json

```
{
  "assets": [
    {
      "asset_code": "USDT.TRC20",
      "coin": "USDT",
      "name": "Tether",
      "network": "TRC20",
      "network_label": "TRON (TRC20)",
      "memo_required": false,
      "memo_type": null,
      "currency": "USDT"
    }
  ]
}
```

```
]
}
```

POST /v1/crypto/collect

Parametre	Type	Statut	Description
amount	number	Requis	Montant brut dans la devise currency
currency	string	Requis	USDT, XAF, XOF, CDF ou USD
asset_code	string	Requis	Reseau retourne par /v1/crypto/assets
reference	string	Optionnel	Reference de commande, generee si absente
notify_url	string	Optionnel	URL HTTPS du webhook marchand
customer	object	Optionnel	firstName, lastName, email
refund_address	string	Optionnel	Adresse de remboursement

Node.js

```
fetch("https://ashtechpay.top/v1/crypto/collect", {
  method: "POST",
  headers: {
    "Authorization": "Bearer YOUR_API_KEY",
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    amount: 25,
    currency: "USDT",
    asset_code: "USDT.TRC20",
    reference: "ORDER-CRYPTO-001",
    notify_url: "https://monsite.com/webhook",
    customer: { firstName: "Ada", lastName: "Lovelace", email: "ada@example.com" }
  })
})
```

json

```
{
  "transaction_id": "8f3e1c2d-...",
  "reference": "ORDER-CRYPTO-001",
  "status": "pending",
  "payment_method": "crypto",
  "asset_code": "USDT.TRC20",
  "network": "TRC20",
  "address": "TX...",
  "memo": null,
  "memo_type": null,
  "amount": 25,
  "currency": "USDT",
  "amount_usdt": 25,
```

```

    "credited_amount": 24.375,
    "fee_amount": 0.625,
    "credited_amount_usdt": 24.375,
    "fee_amount_usdt": 0.625,
    "fee_percent": 2.5,
    "expires_at": "2026-07-31T19:00:00Z"
  }

```

L'API accepte un montant en USDT ou dans une devise fiat supportee (XAF, XOF, CDF ou USD). Les devises fiat sont converties en USDT avec le taux USDT/XAF. amount est le montant brut ; credited_amount_usdt est le net apres frais. Les frais Ashtech Pay et fournisseur sont inclus dans fee_amount_usdt.

Affichage du paiement : la reponse 202 renvoie address, memo, memo_type et asset_code, mais pas une image QR. Generez le QR cote marchand avec une bibliotheque QR a partir de address, affichez l'adresse en texte copiable et affichez toujours le memo/tag dans un champ separe lorsqu'il existe. Pour USDT.TRC20, le QR contient l'adresse ; n'inventez pas de format URI pour un memo dont le format n'est pas documente.

Exemple — afficher adresse, memo et QR (Node.js / navigateur)

```

Node.js

import QRCode from "qrcode";

async function displayCryptoPayment(data) {
  // data vient de POST /v1/crypto/collect
  document.querySelector("#crypto-address").textContent = data.address;
  document.querySelector("#crypto-network").textContent = data.asset_code;
  document.querySelector("#crypto-amount").textContent = data.amount_usdt + " USDT";

  const memoBox = document.querySelector("#crypto-memo");
  if (data.memo) {
    memoBox.textContent = (data.memo_type || "Memo / tag") + " : " + data.memo;
    memoBox.hidden = false;
  } else {
    memoBox.hidden = true;
  }

  // USDT.TRC20 : QR avec l'adresse uniquement
  await QRCode.toCanvas(
    document.querySelector("#crypto-qr"),
    data.address,
    { width: 240, margin: 2, errorCorrectionLevel: "M" }
  );
}

// HTML : #crypto-amount, #crypto-network, #crypto-address,
//         #crypto-qr (canvas) et #crypto-memo

```

Ne concatenez jamais address et memo/tag. Le payeur doit envoyer les fonds sur address avec le memo ou tag exactement tel qu'affiche. Un memo obligatoire oublie peut empecher l'attribution du paiement. Gardez transaction_id et reference, puis attendez payment.completed ou payment.failed avant de livrer.

Apres confirmation par le prestataire crypto, Ashtech Pay credite automatiquement le wallet USDT du marchand avec credited_amount_usdt, puis envoie le webhook POST a notify_url. Le marchand ne doit pas crediter son wallet lui-meme. Les evenements sont payment.completed ou payment.failed et le payload crypto ajoute payment_method, asset_code, address et memo.

Erreurs et diagnostic

Une reponse 502 gateway_error signifie que l'adresse n'a pas pu etre generee. Une reponse 502 provider_invalid_response signifie que le service a repondu sans adresse exploitable. Une reponse 500 server_error contient toujours request_id : conservez-le pour le diagnostic ; ce champ est renvoye dans l'erreur et ne doit pas etre envoye dans la requete. Si provider_status est present, il indique le code HTTP renvoye par le service crypto. Une reponse 400 invalid_email indique que customer.email est mal forme ; invalid_notify_url indique que notify_url doit etre une URL HTTPS valide. Ces deux champs restent optionnels.

5. Initier un paiement — POST /v1/collect

Initie un paiement Mobile Money. Le client recoit une demande de validation sur son telephone. Le routage entre fournisseurs est automatique selon le pays et l'operateur. Les frais sont deduits automatiquement — le champ credited_amount est le montant net credite.

Corps de la requete (JSON)

Parametre	Type	Statut	Description
amount	number	Requis	Montant brut a collecter
currency	string	Requis	Devise du pays (XAF, XOF, CDF...)
phone	string	Requis	Numero de telephone du payeur
operator	string	Requis	Nom exact de l'operateur (depuis /v1/countries)
country_code	string	Requis	Code ISO du pays (CM, SN, CI...)
reference	string	Optionnel*	Reference unique de votre commande. *Obligatoire lors du ret
otp	string	Optionnel	Code OTP recu par SMS. Doit etre accompagne du champ referen
notify_url	string	Optionnel	URL webhook pour recevoir le resultat du paiement

Requete exemple

Node.js

```
fetch("https://ashtechpay.top/v1/collect", {
  method: "POST",
  headers: {
    "Authorization": "Bearer YOUR_API_KEY",
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    amount: 5000,
    currency: "XAF",
    phone: "6700000000",
    operator: "MTN Money",
```

```

country_code: "CM",
reference: "ORDER-001",
notify_url: "https://monsite.com/webhook"
})
})

```

```
curl
```

```

curl https://ashtechpay.top/v1/collect \
-X POST \
-H "Authorization: Bearer YOUR_API_KEY" \
-H "Content-Type: application/json" \
-d '{"amount":5000,"currency":"XAF","phone":"6700000000",
    "operator":"MTN Money","country_code":"CM",
    "reference":"ORDER-001",
    "notify_url":"https://monsite.com/webhook"}'

```

Reponse 202 (succes USSD Push)

```
json
```

```

{
  "transaction_id": "8f3e1c2d-...",
  "reference": "ORDER-001",
  "status": "pending",
  "amount": 5000,
  "credited_amount": 4750,
  "fee_amount": 250,
  "currency": "XAF",
  "operator": "MTN Money",
  "phone": "6700000000",
  "country_code": "CM",
  "created_at": "2026-03-15T14:00:00Z"
}

```

OTP requis — Orange CI/SN/BF (USSD) ou OTP API

Orange CI/SN/BF ! OTP USSD : le serveur retourne un ussd_code a afficher au client (l'OTP s'affiche dans le menu, aucun SMS envoye). Le Mali (Orange Money) ne necessite pas d'OTP — flux standard. Pour un operateur configure en OTP API, le fournisseur envoie automatiquement le SMS et ussd_code = null. Dans les deux cas, la reponse 400 contient un champ 'reference' obligatoire pour l'etape 2.

json

```
// Etape 1 – Requete initiale (sans otp) !'reponse 400
{
  "error": "otp_required",
  "message": "OTP requis. Suivez les instructions de l'opérateur.",
  "reference": "DEP-A1B2C3D4", // !•a conserver absolument
  "ussd_code": null // null=SMS auto | "#144*82#"=USSD a composer
}

// Etape 2 – Retry avec le code reel + reference du 400 !'reponse 202
{
  "amount": 5000, "currency": "XOF", "phone": "07XXXXXXX",
  "operator": "Orange Money", "country_code": "CI",
  "otp": "VOTRE_CODE_OTP", // code reçu ou affiche par l'opérateur
  "reference": "DEP-A1B2C3D4", // !•meme valeur que la reponse 400
  "notify_url": "https://monsite.com/webhook"
}
```

6. Flux de paiement

Selon le pays et l'opérateur, l'API utilise automatiquement l'un des 4 flux ci-dessous. Votre code doit gérer chacun différemment car la réponse et les étapes varient.

Flux	Opérateurs concernés	Réponse initiale	Action requise
USSD Push	MTN, Moov, Airtel, Orange, Free, E-money, Orange Flooz, M-...	202 pending	Attendre le webhook. Le client valide sur son téléphone.
OTP USSD	Orange CI (#144*82#), SN (#144*391#), BIC (#144*4000#), ...	400 otp_required, reference: "DEP-A1B2C3D4", ussd_code: "SSD4182"	Afficher le code USSD, puis relancer avec otp + reference
OTP SMS	Certains opérateurs — SMS automatique	400 otp_required, reference: "DEP-A1B2C3D4", ussd_code: null	Relancer avec otp + reference
Wave	Wave CI, Wave SN	202 pending, flow: wave, wave_url	Afficher le wave_url en bouton ou QR code. Le client ou...

Détection du flux dans votre code

Node.js

```
async function collectPayment(params) {
  const res = await fetch("https://ashtechpay.top/v1/collect", {
    method: "POST",
    headers: { "Authorization": "Bearer YOUR_API_KEY", "Content-Type": "application/json" },
    body: JSON.stringify(params)
  });
  const data = await res.json();

  if (res.status === 202 && data.flow === "wave") {
    // Flux Wave : afficher data.wave_url
    return { type: "wave", waveUrl: data.wave_url, transactionId: data.transaction_id };
  }
  if (res.status === 202) {
    // Flux USSD Push : attendre webhook
  }
}
```

```

    return { type: "ussd_push", transactionId: data.transaction_id };
  }
  if (res.status === 400 && data.error === "otp_required") {
    // Stocker data.reference – obligatoire pour le retry OTP
    if (data.ussd_code) {
      // OTP USSD (Orange CI, SN, BF) : afficher le code a composer
      // CI=#144*82# SN=#144*391# BF=*144*4*6*montant#
      return { type: "otp_ussd", ussdCode: data.ussd_code, reference: data.reference };
    } else {
      // OTP SMS : SMS automatique via le reseau de l'operateur
      return { type: "otp_sms", reference: data.reference };
    }
  }
  throw new Error(data.message);
}

```

7. Statut d'une transaction — GET /v1/transaction/:id

Consultez le statut d'une transaction a tout moment via le transaction_id retourne lors de l'initiation. Vous pouvez utiliser ce endpoint en complement du webhook.

Node.js

```

fetch("https://ashtechpay.top/v1/transaction/8f3e1c2d-...", {
  headers: { "Authorization": "Bearer YOUR_API_KEY" }
})

```

curl

```

curl https://ashtechpay.top/v1/transaction/8f3e1c2d-... \
-H "Authorization: Bearer YOUR_API_KEY"

```

json

```

{
  "transaction_id": "8f3e1c2d-...",
  "reference": "ORDER-001",
  "status": "success",
  "amount": 5000,
  "credited_amount": 4750,
  "fee_amount": 250,
  "currency": "XAF",
  "phone": "6700000000",
  "created_at": "2026-03-15T14:00:00Z",
  "confirmed_at": "2026-03-15T14:02:17Z"
}

```

Statut	Description	Final ?
pending	En attente de confirmation de l'opérateur	Non
success	Paieement confirme — compte marchand credite	Oui
failed	Paieement refuse, expire ou annule	Oui

8. Grille tarifaire en temps reel — GET /v1/fees

Retourne la grille tarifaire en vigueur pour chaque pays actif. Les frais sont configures par l'administrateur et peuvent changer a tout moment. Consultez cet endpoint pour calculer le montant net avant d'appeler /v1/collect.

Node.js

```
fetch("https://ashtechpay.top/v1/fees", {
  headers: { "Authorization": "Bearer YOUR_API_KEY" }
})
```

curl

```
curl https://ashtechpay.top/v1/fees \
-H "Authorization: Bearer YOUR_API_KEY"
```

Reponse

json

```
[
  {
    "country_code": "CM",
    "country_name": "Cameroun",
    "currency": "XAF",
    "deposit_fee_pct": 3.5,
    "withdrawal_fee_pct": 1.5,
    "transfer_fee_pct": 1.0,
    "total_fee_pct": 5.5
  },
  // ...
]
```

Exemple — calculer le montant net avant d'appeler /v1/collect

Node.js

```
// Recuperer les frais en cache (une fois au demarrage ou toutes les heures)
const fees = await fetch("https://ashtechpay.top/v1/fees", {
  headers: { "Authorization": "Bearer YOUR_API_KEY" }
}).then(r => r.json());

// Calculer le montant net credite sur votre compte
function computeNet(grossAmount, countryCode) {
  const fee = fees.find(f => f.country_code === countryCode);
  if (!fee) return grossAmount;
  const feeAmount = Math.round(grossAmount * fee.total_fee_pct / 100);
  return {
    gross: grossAmount,
    fee: feeAmount,
    net: grossAmount - feeAmount,
    fee_pct: fee.total_fee_pct,
  };
}

// Exemple :
console.log(computeNet(10000, "CM"));
// -> { gross: 10000, fee: 550, net: 9450, fee_pct: 5.5 }
```

9. Webhooks

Quand une transaction atteint un etat final, Ashtech Pay envoie automatiquement une requete POST a la notify_url passee dans votre appel a /v1/collect. Le champ amount correspond au montant net apres frais, et total_amount au montant brut collecte. Chaque livraison est dedupliquee, signee en HMAC-SHA256 et retentee automatiquement en cas d'echec temporaire.

Payload — paiement reussi

json

```
{
  "event": "payment.completed",
  "transaction_id": "8f3e1c2d-...",
  "reference": "ORDER-001",
  "status": "completed",
  "amount": 4750, // net apres frais
  "total_amount": 5000, // brut collecte
  "fee_amount": 250,
  "total_fee_amount": 250,
  "currency": "XAF",
  "type": "deposit",
  "phone": "6700000000",
  "timestamp": "2026-03-15T14:02:17.000Z"
}
```

Evenements disponibles

Evenement	Declencheur
payment.completed	Paielement (depot) confirme avec succes
payment.failed	Paielement refuse, expire ou annule
payout.completed	Retrait ou virement sortant confirme
payout.failed	Retrait ou virement echoue

Verifier la signature HMAC

Le corps signe doit etre conserve exactement comme reçu. Calculez HMAC-SHA256 sur timestamp + '.' + corps brut avec votre secret webhook, puis comparez l'en-tete X-Ashtech-Signature. Utilisez X-Ashtech-Event-Id pour dedupliquer les livraisons.

Node.js

```
import crypto from "node:crypto";

app.post("/webhook", express.raw({ type: "application/json" })), (req, res) => {
  const rawBody = req.body.toString("utf8");
  const timestamp = req.header("X-Ashtech-Timestamp") || "";
  const received = req.header("X-Ashtech-Signature") || "";
  const expected = "sha256=" + crypto
    .createHmac("sha256", process.env.ASHTech_WEBHOOK_SECRET)
    .update(timestamp + "." + rawBody)
    .digest("hex");

  const valid = received.length === expected.length &&
    crypto.timingSafeEqual(Buffer.from(received), Buffer.from(expected));
  if (!timestamp || !valid) return res.sendStatus(401);

  const eventId = req.header("X-Ashtech-Event-Id");
  // Ignorez eventId deja traite, puis parsez rawBody et repondez 200.
  const payload = JSON.parse(rawBody);
  return res.status(200).json({ received: true, event: payload.event });
});
```

Handler — Node.js / Express

Node.js

```
app.post("/webhook", express.json(), async (req, res) => {
  // Toujours repondre 200 en premier
  res.status(200).json({ received: true });

  const { event, transaction_id, reference, amount, currency } = req.body;

  if (event === "payment.completed") {
    // amount = montant net (apres frais)
    await markOrderAsPaid(reference, { transactionId: transaction_id, amount, currency });
  }
  if (event === "payment.failed") { await cancelOrder(reference); }
  if (event === "payout.completed") { await markPayoutDone(reference, { transactionId: transaction_id, amount, currency }); }
  if (event === "payout.failed") { await markPayoutFailed(reference); }
});
```

Bonnes pratiques : Verifiez la signature avant tout traitement. Repondez HTTP 200 immediatement, traitez la logique metier de facon asynchrone et dedupliquez avec X-Ashtech-Event-Id ou transaction_id. Votre notify_url doit etre une URL HTTPS publique (pas localhost).

10. Codes d'erreur

En cas d'erreur, l'API retourne un objet JSON avec les champs error et message.

json

```
{
  "error": "bad_request",
  "message": "Champs requis : amount, currency, phone, operator, country_code"
}
```

HTTP	Erreur	Signification
400	bad_request	Parametre manquant ou format invalide
400	otp_required	OTP requis — conservez le champ reference de la reponse, obl
400	missing_reference	Confirmation OTP sans le champ reference — utilisez la valeu
400	otp_expired	Session OTP expiree (15 min) ou introuvable — relancez sans
401	unauthorized	Cle API manquante, invalide ou revoquee
403	forbidden	Cette transaction n'appartient pas a votre compte
404	not_found	Transaction introuvable
422	unprocessable	Pays, operateur ou reseau crypto non supporte / devise incor

422	asset_disabled	Le reseau crypto selectionne n'est plus actif
503	crypto_unavailable	Le catalogue ou le service de paiement crypto est indisponib
429	rate_limited	Trop de requetes — ralentissez
400	invalid_email	customer.email est mal forme
400	invalid_notify_url	notify_url doit etre une URL HTTPS valide
502	gateway_error	Adresse crypto impossible a generer ; consultez message et p
502	provider_invalid_response	Le service crypto a repondu sans adresse exploitable
500	server_error	Erreur interne ; conservez request_id pour le diagnostic

Pour toute question technique non resolue par cette documentation, contactez notre equipe via le support integre a l'application.